AIOT Linux Software Development Guide

版本号: 0.5
发布日期: 2025.06.09

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
0.1	2024.9.25	AWA0511	初始版本
0.2	2024.11.10	AWA0511	刷新 RoadMap
0.3	2025.03.05	AWA0511	增加 A733/T736 编译参考
0.4	2025.03.29	AWA0511	增加 yocto 编译参考 & 刷新 RoadMap
0.5	2025.06.09	AWA0511	刷新 RoadMap& 更新模块开发指引 (recovery/多媒体/图显等)

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 概述 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 读者对象 1
1.4 各个芯片系统支持状态 (持续更新) 1
1.5 2025 RoadMap (持续更新) 2
1.6 2026 RoadMap (持续更新) 2

2 AIOT Linux 产品包 **3**
2.1 简介 3
2.2 AW AIOT 通用软件包获取方法 3
2.2.1 通过代码服务器下载 3
2.3 SDK 适配硬件汇总 (持续更新) 3
2.4 文档资料 3
2.4.1 SDK 文档目录结构 (docs_list.txt) 4
2.5 物料支持列表 4
2.6 SDK 更新 5

3 支持与服务 **6**

4 文档说明 **7**
4.1 Release Note 发布声明 7
4.2 Hardware 硬件类文档 7
4.3 Software 软件类文档 7
4.4 Tools 工具类文档 7
4.5 Product 量产指导文档 8

5 AIOT Linux SDK 软件架构 **9**
5.1 SDK 工程目录介绍 9
5.2 SDK 概述 9
5.2.1 brandy 9
5.2.2 bsp 目录 10
5.2.3 build 10
5.2.4 buildroot 10
5.2.5 debian 11
5.2.6 device 11
5.2.7 kernel 12
5.2.8 openwrt 12
5.2.9 out 12

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2.10 platform 13
5.2.11 prebuilt 14
5.2.12 rtos 14
5.2.13 target 15
5.2.14 tools 15
5.2.15 yocto 15
5.3 SDK 软件框图 15
5.4 SDK 开发流程 16

6 SDK 开发环境搭建 17
6.1 概述 17
6.2 Linux 服务器开发环境搭建 18
6.2.1 发布包使用 Linux 服务器系统版本 18
6.2.2 交叉编译工具链介绍 (持续更新) 19
6.3 Windows PC 开发环境搭建 19
6.3.1 Windows 烧录工具使用 19
6.3.2 USB 驱动安装 20
6.3.3 目标硬件板准备 (持续更新) 21

7 SDK 安装准备工作 22
7.1 概述 22
7.2 安装 REPO 22
7.3 git 配置 23
7.4 SDK 获取 23

8 SDK 编译、打包、烧录 & 升级 24
8.1 SDK 编译 24
8.1.1 模块单独编译 24
8.1.1.1 U-boot 编译 25
8.1.1.2 Kernel 编译 25
8.1.1.3 Buildroot 编译 25
8.1.1.4 Debian 编译 25
8.1.1.5 openwrt 编译 25
8.1.1.6 Yocto 编译 26
8.1.1.7 Recovery 编译 26
8.1.2 全量自动编译 (推荐方式) 26
8.2 SDK 打包 26
8.3 SDK 烧录 & 升级 27
8.3.1 烧录工具 27

9 SDK 开发 28
9.1 uboot 开发 28
9.1.1 概述 28
9.1.2 代码路径 28

版权所有 © 珠海全志科技股份有限公司。保留一切权利

9.1.3 uboot 功能 . . . 28
9.2 kernel 开发 (持续更新) . . . 29
9.2.1 基本配置命令 . . . 29
9.3 Builroot 开发 (持续更新) . . . 29
9.3.1 基本配置命令 . . . 29
9.4 Debian 开发 (持续更新) . . . 29
9.4.1 用户和密码 . . . 30
9.5 Yocto 开发 (持续更新) . . . 30
9.5.1 代码路径 . . . 30
9.5.2 编译环境 . . . 30
9.5.3 编译说明 . . . 31
9.5.3.1 方式一: 原始编译方式 (T527 为例) . . . 31
9.5.3.2 方式二: AW 全量编译方式 (T527 为例) . . . 31
9.5.4 常用配置 . . . 32
9.5.4.1 machine 配置 . . . 32
9.5.4.2 distro 配置 . . . 32
9.5.4.3 image 配置 . . . 33
9.5.5 常用快捷命令 . . . 33
9.6 openwrt 开发 . . . 33
9.7 Recovery 开发 . . . 33
9.8 多媒体开发 . . . 34
9.9 图显开发 . . . 35
9.10 安全机制开发 . . . 36
9.11 应用开发 . . . 36
9.12 WIFI/BT 开发 . . . 36
9.13 TP/Sensor 开发 . . . 36

10 SDK 测试 37

11 SDK 调试 38
11.1 ADB 工具 . . . 38
11.1.1 概述 . . . 38
11.1.2 USB adb 使用说明 . . . 38
11.2 版权检测工具 . . . 38
11.2.1 Buildroot . . . 38
11.2.2 Debian . . . 39
11.2.3 yocto . . . 39

12 常见问题 (FAQ) 40
12.1 如何查看系统相关信息 . . . 40
12.1.1 如何查看系统 Debian 版本 . . . 40
12.1.2 如何查看 Debian 显示使用 X11 还是 wayland . . . 40
12.1.3 如何查看系统分区情况 . . . 40
12.2 Debian deb 包的制作，解压，修改，安装 . . . 41

版权所有 © 珠海全志科技股份有限公司。保留一切权利

12.3 Buildroot 如何对特定软件包进行特定操作 41
12.4 如何新增 yocto 方案 (以 T527 为例) 42

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 2-1　物料库查询路径参考图 4
图 4-1　量产工具获取 8
图 5-1　SDK 开发流程 16
图 6-1　典型开发环境 17
图 6-2　USB 驱动安装 20
图 6-3　USB 驱动安装完成 21
图 9-1　多媒体框架图 35

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 概述

1.1 编写目的

本文档作为 Allwinner AIOT Linux 平台开发指南，内容包含 Allwinner Buildroot/Debian/Yocto/OpenWrt Linux 系统软件开发指南，旨在帮助软件工程师、技术支持工程师快速上手，熟悉 AIOT Linux 软件开发及调试。

1.2 适用范围

AIOT Linux v1.2 及以上版本。

1.3 读者对象

本开发指南适用于 AIOT Linux 软件开发工程师、AIOT 技术支持工程师。

1.4 各个芯片系统支持状态(持续更新)

芯片名称	Buildroot	Debian	Yocto	Kernel	Uboot	SDK	推荐指数
A523/A527/T527	2022.05	N/A	N/A	5.15	2018.07	V1.0	*
A523/A527/T527	2022.05	N/A	N/A	5.15	2018.07	V1.1	*
A523/A527/T527	2022.05	N/A	N/A	5.15	2018.07	V1.2	**
A523/A527/T527	2022.05	11	N/A	5.15	2018.07	V1.3	*
A523/A527/T527	2022.05	11	N/A	5.15	2018.07	V1.4	***
A523/A527/T527	2022.05	11	5.1	5.15	2018.07	V1.4.5	****
A733/T736	2022.05	11	N/A	5.15	2018.07	V1.4.5(alpha)	****

说明

Tina AIOT 暂无规划适配 openwrt，如有需求，可及时联系反馈；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.5 2025 RoadMap (持续更新)

芯片名称	Buildroot	Debian	Yocto	Kernel	Uboot	SDK	预计发布时间
A733/T736	2022.05	11	5.1	5.15	2018.07	V1.4.6(alpha)	2025.Q2
A733/T736	2022.05	11	5.1	5.15	2018.07	V1.4.8(alpha)	2025.Q2
A733/T736	2022.05	11	5.1	5.15	2018.07	V1.5	2025.Q3
A537	2022.05	11	N/A	5.15	2018.07	V1.5.5(alpha)	2025.Q4
A537	2022.05	11	5.1	5.15	2018.07	V1.5.8(alpha)	2025.Q4

1.6 2026 RoadMap (持续更新)

芯片名称	Buildroot	Debian	Yocto	Kernel	Uboot	SDK	预计发布时间
A537	2022.05	11	5.1	5.15	2018.07	V1.6	2026.Q1

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 AIOT Linux 产品包

2.1 简介

AW AIOT Linux 产品包支持 Buildroot，Yocto，Debian 和 openwrt 四个系统，内核基于 Linux5.10、Linux5.15 或 Linux6.6，引导基于 Uboot2018.07 或 Uboot2023.03，适用于 AW EVB 开发板及基于此开发板进行二次开发的所有 Linux 产品。

2.2 AW AIOT 通用软件包获取方法

2.2.1 通过代码服务器下载

Allwinner AIOT Linux SDK 通过全志代码服务器对外发布。客户需要向业务/技术支持窗口申请 SDK 下载权限。申请需同步提供 SSH 公钥进行服务器认证授权，获得授权后即可同步代码。详情下载步骤见全志客户服务平台。

说明

全志客户服务平台https://open.allwinnertech.com。

2.3 SDK 适配硬件汇总 (持续更新)

芯片	类型	架构	板级配置 (device 目录)	方案配置 (target 目录)	备注
A523	通用	arm64	pro3_linux_aiot	pro3_linux_aiot	AW A523 pro3 平板
A527	通用	arm64	pro2_linux_aiot	pro2_linux_aiot	AW A527 pro2 HDMI 平板
T527	通用	arm64	demo_linux_aiot	demo_linux_aiot	AW T527 demo 开发板
A733	通用	arm64	pro3	pro3	AW A733 pro3 平板
T736	通用	arm64	demo_aiot	demo_aiot	AW T736 demo 开发板

2.4 文档资料

AIOT Linux SDK 发布的文档旨在帮助开发者快速上手开发及调试，文档中涉及的内容并不能涵盖所有的开发知识和问题。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

AIOT LINUX 的文档资料来源于两个方面：

- 全志客户服务平台: https://open.allwinnertech.com 资源下载-> 文档该途径的文档实时更新，需要获取最新文档请以此方式
- SDK 源码目录：docs 目录该途径的文档跟随 SDK 版本的迭代更新而更新, 存在一定的滞后性

AIOT Linux SDK 提供丰富的文档资料，包括 Release Note(发布声明)、Hardware(硬件类文档)，Software(软件类文档)，Tools(工具类文档)，Product(量产指导文档)，详情描述见: 第三章节文档说明

说明

文档列表也正在不断更新。如有文档上的疑问及需求，请联系 Allwinner FAE 窗口或访问全志客户服务平台：https://open.allwinnertech.com获取支持。

2.4.1 SDK 文档目录结构 (docs_list.txt)

```
docs/
 ├──── AIOT_Linux_SDK_Note.md
 ├──── AW_AIOT_Linux_Software_Development_Guide_CN.pdf
 ├──── docs_list.txt
 ├──── Product 量产指导文档
 ├──── Release Note 发布声明
 ├──── Software 软件类文档
 ├──── Tools工具类文档
 └──── Socs
```

2.5 物料支持列表

请以全志客户服务平台https://open.allwinnertech.com物料库为准，当前提供 EMMC，DRAM，SLC NAND，MLC NAND，SPI NAND，NOR，WIFI/BT，以太网 PHY 支持列表。

物料库路径参考：

图 2-1: 物料库查询路径参考图

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.6 SDK 更新

SDK 更新分为两类：

- 1. 以补丁的形式发布到一号通，发布后系统以邮件形式通知开发者；
- 2. 定期（半年或季度）的小版本迭代升级，将过往的补丁合入到 SDK 中，发布后以系统邮件通知开发者，可基于干净的 SDK 包通过 repo sync 命令更新。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 支持与服务

- Allwinner 提供全志客户服务平台: https://open.allwinnertech.com，用来登记客户遇到的问题以及解决状态。方便双方追踪，使问题处理更加高效。后续 SDK 问题、技术问题、技术咨询等都可以提交到此系统上，Allwinner 技术服务会及时将问题进行分发、处理和跟踪。
- 如有文档上的疑问及需求，也可通过我们的文档窗口进行反馈: aw-document@allwinnertech.com

说明

系统登录帐号需要与 Allwinner 开通确认。

广州星翼电子科技有限公司zdyzlinux

4 文档说明

AIOT Linux SDK 提供丰富的文档资料，包括 Release Note(发布声明)、Hardware(硬件类文档)，Software(软件类文档)，Tools(工具类文档)，Product(量产指导文档)，通过全志客户服务平台: https://open.allwinnertech.com获取，如无权限方法，请联系对应业务和支持窗口提供解决方案或者直接通过全志客户服务平台进行申请。

4.1 Release Note 发布声明

包含文档 list 指引和 SDK 版本发布说明，详情见各个目录下的文档。

4.2 Hardware 硬件类文档

包含芯片手册、硬件设计指南，原理图，PCB，部分硬件物料清单支持列表，详情见各个目录下的文档。

4.3 Software 软件类文档

包含通用模块开发指南、基础组件开发指南、差异化开发指南、软件认证指南，详情见各个目录下的文档。

4.4 Tools 工具类文档

工具需要以全志客户服务平台工具https://open.allwinnertech.com为准，工具对应使用文档默认集成在工具安装目录。

工具及其文档获取参考路径如下，如依然存在疑问请联系 AW 对应业务和支持人员。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

图 4-1: 量产工具获取

工具介绍可以参考：文档指南/Tools 工具类/《全志工具包介绍.pdf》

4.5 Product 量产指导文档

包含通用接口的问题指导文档、产线流程参考、产线问题快速排查等文档，详情见各个目录下的文档。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 AIOT Linux SDK 软件架构

5.1 SDK 工程目录介绍

AIOT Linux SDK 目录结构, 主要有构建工具、构建系统、配置工具、工具链、芯片配置目录、内核及 boot 目录等组成, 下面按照目录顺序做简单介绍。

```
AIOT-LINUX/
    ├── brandy :存放BootLoader(boot0/uboot)、arisc相关代码
    ├── bsp : bsp目录下存放着Allwinner的驱动，以及对内核的改动，保证着原生内核(kernel/linux-X.X)比较干净的状态，
      并且能够快速移植适配到新内核中。
    ├── build : 存放AIOT Linux系统构建脚本
    ├── buildroot： Buildroot相关的配置文件以及原生的buildroot代码，支持部分芯片
    ├── build.sh -> build/top_build.sh : 编译构建入口脚本文件
    ├── debian : debian相关配置文件以及原生的debian构建代码，支持部分芯片
    ├── device : 存放芯片方案的板级配置文件
    ├── kernel： 存放不同版本的内核代码
    ├── openwrt： 存放着OpenWrt原生代码，支持部分芯片
    ├── platform： 存放着一些通用的软件包源码，包括aw(多媒体、网络等)和第三方的(lvgl、stressapp等)
    ├── prebuilt： 存放交叉编译工具链
    ├── rtos： 存放异构系统构建环境，不同平台rtos构建存在差异
    ├── target： 对应device目录，存放板卡，芯片的业务逻辑配置
    ├── tee_kit： tee相关demo和配置，支持部分芯片
    ├── tools： tools目录用于存放一些host端工具, 例如打包工具
    └── yocto： yocto相关配置文件以及原生的yocto构建代码，支持部分芯片
```

说明

1. **部分 SDK 下载，可能不存在对应的目录，这表明该芯片未支持该部分, 请参考芯片系统支持情况，如有需求，可及时联系反馈；**

5.2 SDK 概述

5.2.1 brandy

brandy 目录下主要存放 BootLoader(boot0，uboot), arisc，dram 库等代码。

- 方式一:

```
brandy
    ├── arisc //cpus 代码,通常用于休眠唤醒,调频调压
    ├── brandy-2.0
    │   ├── build.sh -> tools/build.sh
    │   ├── opensbi
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    ├──── spl-pub
    │   ├──── tools
    │   └──── u-boot-2018
    └──── dramlib
```

- 方式二: 独立 bsp 仓库机制

```
brandy
    ├──── arisc
    ├──── brandy-2.0
    │   ├──── build.sh -> tools/build.sh
    │   ├──── opensbi
    │   ├──── tools
    │   ├──── u-boot-2023
    │   └──── u-boot-bsp
    └──── dramlib
```

说明

每个芯片 SDK 包使用的 uboot 版本可能不一样，通常只支持及提供一个版本的 uboot，请以实际 sdk 中 uboot 为准。u-boot-2023 使用了独立 bsp 仓库机制，即 u-boot-2023 目录下是原生 uboot，而 Allwinner 相关改动内容都放在 u-boot-bsp 目录中，编译时会自动做处理合成一体。

5.2.2 bsp 目录

bsp 目录下存放着 Allwinner 的驱动，以及对内核的改动，保证着原生内核 (kernel/linux-X.X) 比较干净的状态，并且能够快速移植适配到新内核中。

5.2.3 build

build 目录存放 AIOT Linux 的系统构建脚本，主要功能有：

1. 提供编译需要的环境变量、函数、规则。
2. 提供各目标模块的编译方法、规则。
3. 对接 OpenWrt, Buildroot 等不同构建系统。
4. 打包生成系统固件的脚本。

5.2.4 buildroot

AW AIOT Linux SDK 的 Buildroot 系统，其包含了基于 Linux 系统开发用到的各种系统源码，驱动，工具，应用软件包。Buildroot 是 Linux 平台上的一个开源的嵌入式 Linux 系统自动构建框架。整个 Buildroot 是由 Makefile 脚本和 Kconfig 配置文件构成的。你可以通过 Buildroot 配置，编译出一个完整的可以直接烧写到机器上运行的 Linux 系统软件

Buildroot 有以下几点优势：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 通过源码构建，有很好的灵活性；
- 方便的交叉编译环境，可以快速构建；
- 方便的各系统组件配置及定制开发。

使用 Buildroot 的 project 最出名的就是 Openwrt 。可以看到，由它制作出的镜像可以跑在搭载 16 Mb SPI NOR 的路由器上，系统基本没包含多余的东西。这就是得益于 Buildroot 的地方：简单化。整个 Buildroot project 在一个 git 维护。

Buildroot 使用 kconfig 和 make，一个 defconfig 配置代表一种 BSP 支持。Buildroot 本身不具备扩展能力，用户需要自己通过脚本来完成定制。这些列出来的特点，都是和 Yocto 不同的地方。

说明

- **部分 SDK 可能没有 buildroot 目录，即不支持 buildroot。具体参考芯片系统支持情况。**
- **详情的 buildroot 介绍，请参考文档：Software 软件类文档/基础组件开发指南/buildroot/《Tina_Linux_buildroot_ 使用指南.pdf》**

5.2.5 debian

Debian 是个自由，开源的 Linux 操作系统。详情可见：https://www.debian.org

```
debian/
├── build.sh
├── compressed_files
├── debian-build-service
├── mk-base-debian.sh
├── mk-image.sh
├── overlay
├── packages
├── readme.md
└── tools
```

说明

- **部分 SDK 可能没有 debian 目录，即不支持 debian。具体参考芯片系统支持情况。**
- **关于 AW 的 Debian 使用适配，可参考文档：Software 软件类文档/基础组件开发指南/Debian/《AW_AIOT_Linux_debian_ 开发指南》**

5.2.6 device

device 目录用于存放芯片方案的配置文件，包括内核配置，env 配置，分区表配置，sys_config.fex，board.dts 等。

```
device
├── config
│   ├── chips
│   ├── common
│   └── rootfs_tar
└── product -> ./config/chips/t527
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

rootfs_tar 中存放着 rootfs 目录包，当使用 BSP 编译方式时，使用到的预制 rootfs 就是从这里制作而成。

5.2.7 kernel

kernel 目录主要存放不同版本的内核代码。

```
kernel
  ├──── linux-4.9
  ├──── linux-5.10
  └──── linux-5.15
```

说明

- **Allwinner 都会采用独立 bsp 仓库方式，即 kernel/linux-X.X 目录存放的是原生内核，而 Allwinner 的驱动都会存放在 bsp 目录下，如bsp 目录描述。具体参考芯片系统支持情况。**

5.2.8 openwrt

openwrt 目录存放着 OpenWrt 原生代码，及软件包、芯片方案目录。

```
openwrt
  ├──── build      # OpenWrt构建系统相关hook脚本
  ├──── dl         # 软件压缩包
  ├──── openwrt    # OpenWrt原生代码目录
  └──── package    # 额外添加的软件包,包括AW软件包、第三方软件包，feeds引入的软件包
```

说明

- **部分 SDK 可能没有 openwrt 目录，即不支持 openwrt。具体参考芯片系统支持情况**

5.2.9 out

out 目录用于保存编译相关的临时文件和最终镜像文件，编译后自动生成此目录，例如编译方案 demo_linux_aiot 。

```
out
  ├──── demo_linux_aiot
  │   ├──── demo
  │   │   ├──── buildroot
  │   │   ├──── openwrt
  │   │   ├──── bsp
  │   │   ├──── debian
  │   │   ├──── yocto
  │   │   └──── pack_out
  │   └──── kernel
  └──── toolchain
      └──── gcc-linaro-5.3.1-2016.05-x86_64_arm-linux-gnueabi
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

根据配置选择，最终的编译产物会在对应 openwrt/buildroot/bsp/debian/yocto 目录下，而打包生成固件的准备文件都存放在 pack_out 下，kernel 是内核编译产物 (仅限 linux-5.4 及之后版本)，toolchain 目录是存放着解压后的工具链，用于编译内核。

- **如 openwrt 目录**

```
openwrt
  ├── boot.img
  ├── rootfs.img
  ├── build_dir
  ├── staging_dir
  ├── tmp
  ├── extra
  ├── r528_linux_evb4_uart0.img
  ├── uImage
  ├── vmlinux
  └── zImage
```

- boot.img 为最终烧写到系统 boot 分区的数据，可能为 boot.img 格式也可能为 uImage 格式。
- rootfs.img 为最终烧写到系统 rootfs 分区的数据，该分区默认为 squashfs 格式。
- build_dir 为 sdk 编译 host，target 和 toolchain 的临时文件目录，存有各个软件包的源码。
- staging_dir 为 sdk 编译过程中保存各个目录结果的目录。
- extra 目录下会保存的是最终生成的 ipk 软件包。
- tmp 目录下会保存着本方案软件包相关的信息。
- r528_linux_evb4_uart0.img 为最终固件包 (系统镜像)，串口信息通过串口输出。
- 若使用 pack -d，则生成的固件包为 xxx_card0.img，串口信息转递到 tf 卡座输出。

5.2.10 platform

platform 目录存放着一些软件包源码，这些软件包的编译方式是通用的，分别可以用在 OpenWrt、Buildroot、debian, yocto 等不同构建系统中。这个目录的存在是为了不同构建系统共用软件包提供可能性。

目前 platform 目录结构如下，主要根据是否 AW 自研改动，以及类别区分：

```
platform
  ├── allwinner
  │   ├── power
  │   ├── system
  │   ├── usb
  │   ├── utils
  │   └── wireless
  └── thirdparty
      └── gui
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2.11 prebuilt

prebuilt 目录存放着一些预编译好的工具：

```
├── hostbuilt
│   ├── make4.1
│   └── python3.8
├── kernelbuilt
│   ├── aarch64
│   ├── arm
│   └── riscv
└── rootfsbuilt
    ├── aarch64
    ├── arm
    └── riscv
```

- hostbuilt 目录下存放着 make 以及 python3.8 工具，为了解决个别 Host 端工具版本过低导致编译失败的问题。
- kernelbuilt 目录存放着编译内核的工具链压缩包，它会被解压到 out/toolchain 目录下。
- rootfsbuilt 目录存放着编译 rootfs 的工具链。

5.2.12 rtos

rtos 目录用于存放异构系统构建环境，不同平台 rtos 构建存在差异，rtos 构建查看《FreeRTOS_RTOS_ 系统 _ 开发指南》、《{IC}_RTOS_ 系统 _ 开发指南》：

```
├── board
│   └── mr527_e906
├── envsetup.sh -> tools/scripts/source_envsetup.sh
├── lichee
│   ├── dsp
│   ├── rtos
│   ├── rtos-components
│   └── rtos-hal
└── tools
    ├── image-file
    ├── scripts
    ├── tool
    ├── win-tools
    └── xradio-tools
```

- board：板级配置目录，用于存放芯片方案的配置文件，主要包括系统端口配置文件 sys_config.fex 等。
- lichee/dsp：存放 DSP FreeRTOS 系统、组件、应用。
- lichee/rtos：存放 FreeRTOS 系统、组件、应用。
- lichee/rtos-components：公共组件目录，lichee/dsp 与 lichee/rtos 都可以使用该组件。
- lichee/rtos-hal：BSP 驱动目录，用于存放各种驱动代码。对 lichee/dsp 与 lichee/rtos 通用。
- tools：工具目录，用于存放编译打包相关的脚本、工具等。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2.13 target

target 目录对应这 device 目录内容。本目录主要存放不同 OS 对应不同板卡，不同芯片的业务逻辑配置。

```
target/
├── a523
│   ├── buildroot
│   │   └── pro3_linux_aiot
│   ├── debian
│   └── pro3_linux_aiot
├── a527
│   ├── buildroot
│   │   └── pro2_linux_aiot
│   └── debian
│       └── pro2_linux_aiot
├── common
└── t527
    ├── buildroot
    │   └── demo_linux_aiot
    └── debian
        └── demo_linux_aiot
```

5.2.14 tools

tools 目录用于存放一些 host 端工具, 例如打包工具。

5.2.15 yocto

Yocto 和 Buildroot 一样，是一套构建嵌入式系统的工具，但是两者的风格完全不同。

```
yocto/
├── build
├── dl
├── envsetup.sh -> sources/base/envsetup.sh
├── setup-environment -> sources/base/setup-environment
└── sources
```

说明

部分 SDK 或 SDK 版本可能没有 yocto 目录, 即不支持 yocto。具体参考芯片系统支持情况。

5.3 SDK 软件框图

待补充

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.4 SDK 开发流程

AW AIOT LINUX 系统是基于 Buildroot/Yocto/Debian/OpenWrt 系统，内核基于 kernel 4.9/5.10/5.15 开发，针对多种不同的产品形态开发的 SDK。客户可以基于此 SDK，有效的实现系统定制和对应产品开发工作。

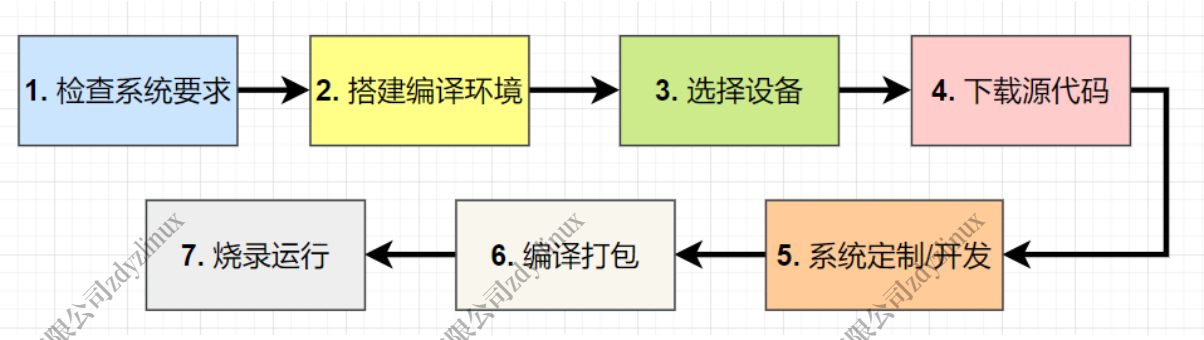

图 5-1: SDK 开发流程

如上图所示，开发者遵循上述开发流程，在本地快速构建 AW AIOT LINUX 系统的开发环境和编译代码。简述流程如下：

- 1. 检查系统要求：在下载代码和编译前，需确保本地的开发设备能够满足需求，包括机器的硬件能力，软件系统，工具链等。目前 SDK 支持 Linux 操作系统环境下编译，并仅提供 Linux 环境下的工具链支持，其他如 MacOS，Windows 等系统暂不支持。
- 2. 搭建编译环境：介绍开发需要安装的各种软件包和工具，详情见SDK 开发环境搭建
- 3. 选择设备：开发者根据自己的开发需求，选择对应的硬件版型，详情见SDK 适配硬件汇总
- 4. 下载源代码：选择硬件版型后，需要安装 repo 工具用于批量下载源代码，详情见SDK 安装准备工作
- 5. 系统定制：开发者根据选定的硬件板卡，产品定义，定制开发，详情见SDK 开发
- 6. 编译打包：开发者根据选定的硬件进行编译，打包生成固件 img. 详情见SDK 编译
- 7. 烧录运行：生成固件 img 之后，开发者通过烧录固件或者烧录方式将固件 img 烧录并运行在硬件板卡中，详情见SDK 烧录 & 升级

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 SDK 开发环境搭建

6.1 概述

本节主要介绍如何本地搭建编译环境来编译 Allwinnertech AIOT Linux SDK 源代码。当前 SDK 只支持在 Linux 环境下编译，并提供 Linux 下的交叉编译工具链。一个典型的嵌入式开发环境通常包括 Linux 服务器，Windows/MAC/Linux PC 和目标硬件板，典型开发环境如下图所示。

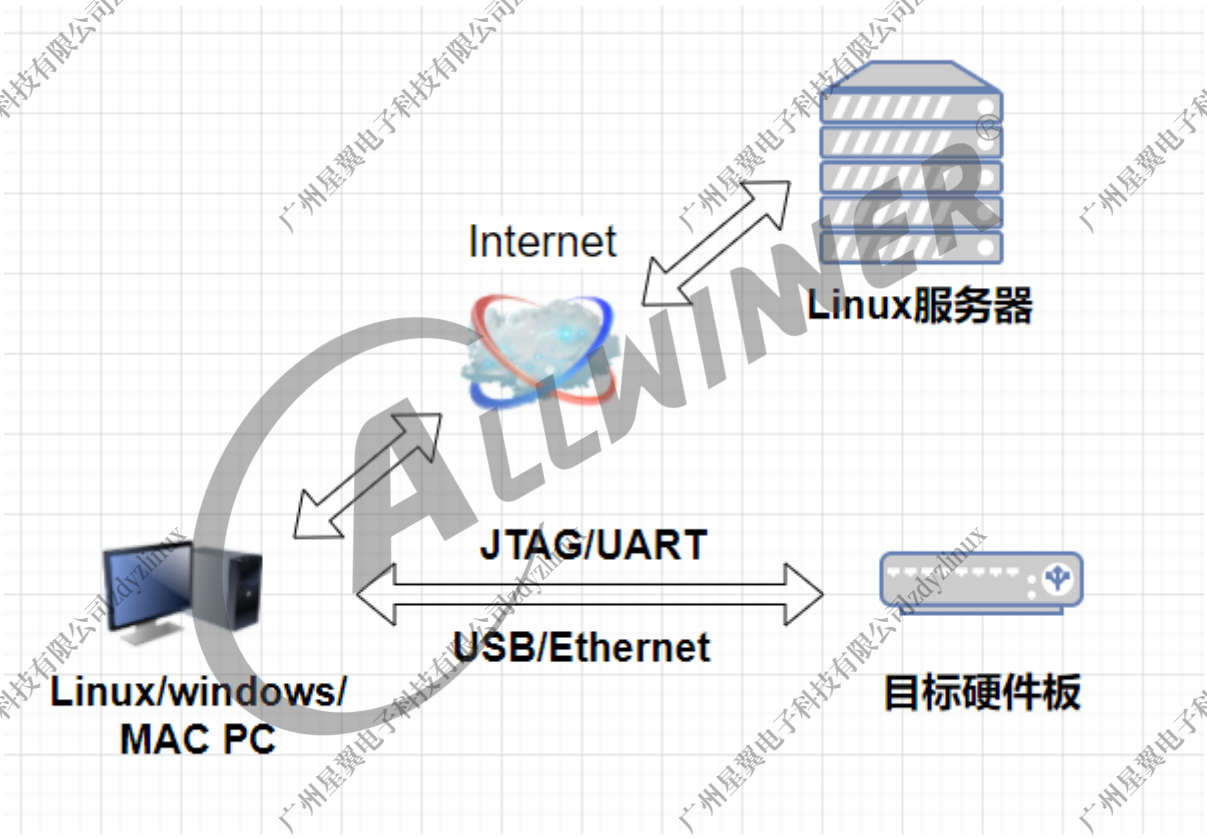

图 6-1: 典型开发环境

- Linux 服务器上建立交叉编译环境，为软件开发提供代码更新下载和交叉编译服务
- Windows/MAC/Linux PC 和 Linux 服务器共享程序，并安装 MobaXterm 等工具，通过网络远程登陆到 Linux 服务器，进行交叉编译、代码开发调试等工作
- Windows/MAC/Linux 通过串口或者 USB 与目标硬件板连接，将编译后的镜像 (img) 烧写到目标硬件板，并进行调试工作。

说明

- **AW 提供 linux/mac/windows 上的烧录工具，客户可以根据研发习惯进行自行选择。**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.2 Linux 服务器开发环境搭建

我们推荐使用 Ubuntu 20.04 的系统进行编译。其它 linux 版本可能需要对软件包做相应调整。除了系统要求外，还有其它硬件方面的要求。

- 硬件要求：64 位系统，硬盘建议大于 128G。
- 软件要求: Ubuntu 20.4 系统

编译 SDK 环境搭建所依赖的软件包安装命令如下：

AIOT Linux SDK 是在 ubuntu14.04/16.04/18.04/20.04/22.04 下均测试验证过可正常编译开发。其他版本可能需要对软件包做相应调整。

编译 AIOT Linux SDK 之前，需要先确定编译服务器安装了 gcc，binutils，bzip2，flex，python，perl，make，ia32-libs，find，grep，diff，unzip，gawk，getopt，subversion，libz-dev，libc headers 等软件包。

ubuntu 14.04 版本可直接执行以下命令安装：

```
sudo apt-get install build-essential subversion git-core libncurses5-dev zlib1g-dev gawk flex quilt libssl-dev xsltproc
    libxml-parser-perl mercurial bzr ecj cvs unzip lib32z1 lib32ncurses5 lib32z1-dev lib32stdc++6 libstdc++6 ncurses-
    term bison libexpat1-dev -y
```

ubuntu 16.04/18.04 版本，执行下面命令安装软件包：

```
sudo apt-get install build-essential subversion git-core libncurses5-dev zlib1g-dev gawk flex quilt libssl-dev xsltproc
    libxml-parser-perl mercurial bzr ecj cvs unzip lib32z1 lib32z1-dev lib32stdc++6 libstdc++6 libc6:i386 libstdc++6:i386
    lib32ncurses5 lib32z1 ncurses-term bison libexpat1-dev -y
```

ubuntu 20.04/22.04 版本，执行下面命令安装软件包：

```
sudo apt-get install build-essential subversion git-core libncurses5-dev zlib1g-dev gawk flex quilt libssl-dev xsltproc
    libxml-parser-perl mercurial bzr ecj cvs unzip lib32z1 lib32z1-dev lib32stdc++6 libstdc++6 libc6:i386 libstdc++6:i386
    lib32ncurses-dev lib32z1 ncurses-term bison libexpat1-dev -y
```

- **建议使用 Ubuntu20.04 系统或更高版本开发，若编译遇到报错，可以视报错信息，安装对应的软件包。**

说明

考虑到客户开发环境搭建时间成本问题，allwinner 也将会提供了交叉编译器 docker 进行方式供客户验证，缩短编译环境搭建耗时。后续将发布参考文档《AW_AIOT_Linux_Developer_Guide_Docker_Deploy_CN.pdf》，该项支持规划中，预计 2025.Q3 完成

6.2.1 发布包使用 Linux 服务器系统版本

本 SDK 开发环境按照如下版本 Linux 系统，SDK 默认均以此 Linux 系统进行编译：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
Distributor ID: Ubuntu
Description:    Ubuntu 20.04.6 LTS
Release:        20.04
Codename:       focal
Linux version 5.15.0-113-generic (buildd@lcy02-amd64-017) (gcc (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0, GNU ld (GNU
    Binutils for Ubuntu) 2.34) #123~20.04.1-Ubuntu SMP Wed Jun 12 17:33:13 UTC 2024
```

6.2.2 交叉编译工具链介绍 (持续更新)

鉴于 AW AIOT Linux SDK 目前只在 Linux PC 环境下编译, 我们也仅提供 Linux 下的交叉编译工具链。其中工具链预制目录在 prebuilt 下。

Uboot 编译工具链:

```
Uboot2018 SDK:

brandy/brandy-2.0/tools/toolchain/gcc-linaro-7.2.1-2017.11-x86_64_arm-linux-gnueabi.tar.xz
```

kernel 编译工具链:

```
Linux5.15 SDK:

prebuilt/kernelbuilt/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.xz
```

Buildroot 内置交叉编译:

```
buildroot-202205 SDK:

buildroot/buildroot-202205/dl/toolchain-external*
```

说明

buildroot 编译构建在持续优化中，如有意见和建议，请及时反馈。

6.3 Windows PC 开发环境搭建

windows 开发环境搭建主要是介绍如何将固件 img 烧写到目标硬件板卡中。

6.3.1 Windows 烧录工具使用

AW 在 windows 提供 USB 升级工具 (PhoenixSuit) 和 USB 量产工具 (PhoenixUSBpro);

- USB 升级工具 (PhoenixSuit): 单设备的固件烧写升级工具，支持烧录单分区，保留数据升级，分区擦除升级，全盘擦除升级、指定地址烧录等功能；工具多用于研发阶段。
- USB 量产工具 (PhoenixUSBpro)：多设备并行的固件烧录升级工具，最大支持 24 路 (默认 8 路) 同时烧录；工具多用于产线生产。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

- **受限于 windows PC USB 性能的影响，建议产线生产使用 8 路。如需使用 16 路/24 路工具，请从 AW FAE 处获取。**
- **工具的详细使用介绍请参考相关工具文档。**

6.3.2 USB 驱动安装

升级烧录设备的识别需要安装 AW usb 驱动。驱动存放在工具的安装目录下:Drivers/AW_Driver。

图 6-2: USB 驱动安装

版权所有 © 珠海全志科技股份有限公司。保留一切权利

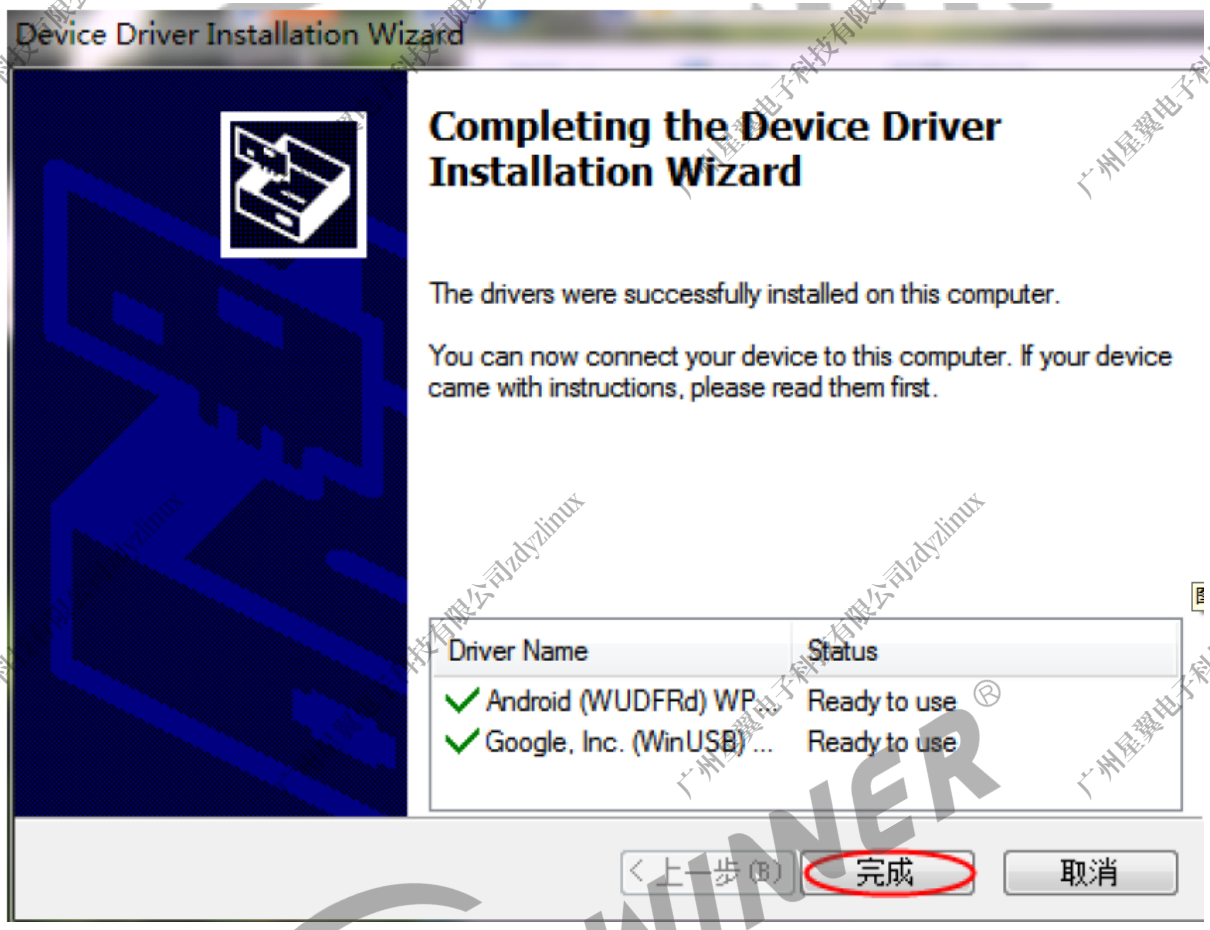

图 6-3: USB 驱动安装完成

说明

正常情况下，工具的安装过程会自动安装 USB 驱动，如果未安装请主动安装。

6.3.3 目标硬件板准备 (持续更新)

请参考适配硬件汇总, 选择对应硬件进行开发调试。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7 SDK 安装准备工作

7.1 概述

Allwinner AIOT Linux SDK 的代码是通过多个 git 仓库进行版本控制和管理的。开发者可以通过使用 repo 对这些 git 仓库进行统一的下载、提交等操作。

7.2 安装 REPO

确保主目录下有一个 bin/ 目录，并且该目录包含在路径中：

```
mkdir ~/bin
export PATH=~/bin:$PATH
```

下载 repo 工具

- 方式一

通过访问 google 地址下载 repo 工具

```
curl https://storage.googleapis.com/git-repo-downloads/repo > ~/bin/repo
chmod a+x ~/bin/repo
```

- 方式二

通过访问国内的站点来下载 repo 工具

```
curl https://mirrors.tuna.tsinghua.edu.cn/git/git-repo -o ~/bin/repo
chmod a+x ~/bin/repo
```

- 方式三

使用如下命令获取 repo

```
sudo apt-get install repo
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7.3 git 配置

一般在使用 repo 之前，需要配置 git 的信息，否则后面的操作可能会遇到 hook 检查的问题 (当然这里是提前配置 git 信息，也可遇到问题时，根据提示进行操作)

```
git config --global user.name "your name"
git config --global user.email "your mail"
```

7.4 SDK 获取

获取方式请参考: AW AIOT 通用软件包获取方法

版权所有 © 珠海全志科技股份有限公司。保留一切权利

8 SDK 编译、打包、烧录 & 升级

8.1 SDK 编译

- 本 SDK 开发环境是在 Ubuntu 上开发测试。我们推荐使用 Ubuntu 22.04 的系统进行编译。其它 Linux 版本可能需要对软件包做相应的调整，参考SDK 开发环境搭建
- 编译方式可分为：

1. **模块单独编译**
2. **全量自动编译**

8.1.1 模块单独编译

在进行模块编译之前，需要配置相关信息，如下：

```
./build.sh config
```

根据需要选择配置，例如依次选择 linux,buildroot,t527,demo-aiot-linux,default,linux-5.15

具体支持情况需要参考芯片系统支持情况

```
$ ./build.sh config
========ACTION List: mk_config ;========
options :
All available platform: //选择linux系统
  0. android
  1. linux
Choice [linux]:1
All available linux_dev: //选择需要的OS
  0. bsp
  1. dragonboard
  2. buildroot
  3. debian
  4. yocto
Choice [buildroot]:2
All available ic: //选择需要的芯片平台
  0. a523
  1. a527
  2. a733
  3. t527
  4. T736
Choice [t527]: 3
All available board: //选择需要的板卡方案
  0. demo_linux_aiot
  1. demo_linux_aiot_nand
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
  2. demo_linux_aiot_spinand
Choice [demo_linux_aiot]:0
All available flash:  //选择需要的存储介质
  0. default
  1. nor
Choice [default]:0
All available kern_name: //选择需要的内核版本
  0. linux-5.10
  1. linux-5.15
Choice [linux-5.15]:1
Setup BSP files
```

说明

1. 详细可查看脚本逻辑:build/mkcommon.sh 和 build/mkcmd.sh

2. 具体选择配置请务必参考实际的参考硬件适配情况 和芯片系统支持情况

8.1.1.1 U-boot 编译

```
./build.sh bootloader
```

8.1.1.2 Kernel 编译

```
./build.sh kernel
```

8.1.1.3 Buildroot 编译

```
./build.sh buildroot_rootfs
```

8.1.1.4 Debian 编译

```
./build.sh debian_rootfs
```

8.1.1.5 openwrt 编译

```
./build.sh openwrt_rootfs
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

8.1.1.6 Yocto 编译

```
./build.sh yocto_rootfs
```

8.1.1.7 Recovery 编译

待补充

8.1.2 全量自动编译 (推荐方式)

- 1. 选择方案

```
./build.sh config
```

根据需要选择配置，例如依次选择 linux,buildroot,t527,demo-aiot-linux,default,linux-5.15

说明

具体配置选择与单独编译的配置类似, 如描述

- 2. 执行编译

```
./build.sh
```

8.2 SDK 打包

在上述完成编译之后，进入工程目录执行以下命令即可打包生成对应固件。

- 打包 normal 固件

```
./build.sh pack
```

- 打包安全固件

```
./build.sh pack_secure
```

- 打包 normal card0 固件

```
./build.sh pack_card
```

- 打包安全 card0 固件

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
./build.sh pack_secure_card
```

8.3 SDK 烧录 & 升级

本章节主要介绍如何将构建完成的镜像文件 (image) 烧写并运行在硬件设备上的流程。

SDK 中的烧录工具不再更新，后续会删除，请优先选择从全志客户服务平台下载最新烧录工具。

windows 工具均集成在 APST 中，下载安装 APST 即可，APST 的工具均自带文档。

8.3.1 烧录工具

AIOT Linux 提供的几种镜像烧写工具介绍如表所示，用户可以选择合适的烧写方式进行烧写。

工具	运行系统	描述
PhoenixSuit	windows	分分区升级及整个固件升级工具
PhoenixCard	windows	卡固件制作工具
PhoenixUSBpro	windows	量产升级工具，支持 USB 一拖 8 烧录
LiveSuit	ubuntu	固件升级工具

对于 ubuntu：

- 64bit 主机使用 LiveSuitV306_For_Linux64.zip。
- 32bit 主机使用 LiveSuitV306_For_Linux32.zip。

具体烧录工具和使用说明，请到全志客户服务平台下载。

说明

如需使用烧录器方式，请参考文档《Linux_MMC_NOR 离线烧录 _ 开发指南.pdf》。

说明

进入烧录模式的方法存在几种，如下描述：

1. **BROM 无法读取到 boot0，例如新换的 flash 不包含数据，或者上电时短路 flash 阻断通信。**
2. **在串口中按 2 进入烧录。即，在串口工具输出框中，按住键盘的’2’，不停输出字符’2’，上电启动。boot0 检测到此字符，会跳到烧录模式。**
3. **在 uboot 控制台，执行 efex。**
4. **在 linux 控制台，执行 reboot efex。**
5. **adb 可用的情况下，可使用 adb shell reboot efex，或点击烧录工具上的 “立即烧录” 按钮。**
6. **当完整配置 [fel_key] 下 fel_key_max 和 fel_key_min 时，按下键值在范围内按键，之后上电。**
7. **当板子有 FEL 按键时，按住 FEL 按键上电。**
8. **制作特殊的启动卡，从卡启动再进入烧录模式。**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

9 SDK 开发

9.1 uboot 开发

9.1.1 概述

本章节简单介绍 uboot 基本配置、功能裁剪、编译打包、常用命令的使用，帮助客户了解 AIOT LINUX 平台 uboot 框架，为 boot 定制开发提供基础。

目前 AIOT Linux SDK 默认使用 uboot-2018。

9.1.2 代码路径

```
AIOT-LINUX/
    ├──── brandy
    │     └──────── brandy-2.0
    │        └──── u-boot-2018
```

9.1.3 uboot 功能

AIOT Linux SDK 中，bootloader(boot0/uboot) 在内核运行之前运行，可以初始化硬件设备、建立内存空间映射图，从而将系统的软硬件环境带到一个合适状态，为最终调用 linux 内核准备好正确的环境。在 AIOT LINUX 系统平台中，除了必须的引导系统启动功能外，uboot 还提供烧写、升级等其它功能。

- 引导内核能从存储介质（nand/mmc/spinor）上加载内核镜像到 DRAM 指定位置并运行。
- 量产 & 升级包括卡量产，USB 量产，私有数据烧录，固件升级。
- 电源管理包括进入充电模式时的控制逻辑和充电时的显示画面。
- 开机提示信息开机能显示启动 logo 图片 (BMP 格式)。
- Fastboot 功能实现 fastboot 的标准命令，能使用 fastboot 刷机。

说明

关于更为详尽的 uboot 的开发介绍，请参考文档《Linux_U-Boot_ 开发指南.pdf》。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

9.2 kernel 开发 (持续更新)

9.2.1 基本配置命令

```
# 进行内核方案配置选择
./build.sh config

# kernel配置
./build.sh menuconfig

#kernel配置保存
./build.sh saveconfig
```

9.3 Builroot 开发 (持续更新)

AW 已经配置好环境变量、BSP 配置和各个模块的开发，方便客户开发定制。

9.3.1 基本配置命令

```
# 进行buildroot方案配置选择
./build.sh config

# buildroot配置
./build.sh buildroot_menuconfig

#buildroot配置保存
./build.sh buildroot_saveconfig
```

说明

详情的开发指导请参考文档：文档中心/Software 软件类文档/基础组件开发指南/《Tina_Linux_buildroot_ 使用指南》

9.4 Debian 开发 (持续更新)

Aw 提供了对 Debian 11 文件系统的支持，同时兼容了 x11 以及 wayland 两种不同的显示架构。添加了 GPU 加速等 package。

Debian 开发文档参考：

```
文档中心/Software软件类文档/基础组件开发指南/debian/《AW_AIOT_Linux_debian_开发指南.pdf》
```

基于 Debian 部署 docker 开发文档参考：

```
文档中心/Software软件类文档/基础组件开发指南/debian/《AW_Deveoper_guide_debian_docker_CN.pdf》
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

9.4.1 用户和密码

用户：root

密码：root

9.5 Yocto 开发 (持续更新)

Yocto 项目是一个开源的合作项目，旨在通过提供标准化的构建工具和流程，简化并加速基于 Linux 的嵌入式操作系统的开发。它为开发者提供了高度灵活、模块化的环境，以构建针对特定硬件优化的定制化 Linux 发行版。

9.5.1 代码路径

```
AIOT-LINUX/
  └──── yocto
    ├──── build  #编译过程目录
    │    ├──── cache
    │    ├──── conf #配置文件，如bblayers.conf(来源于base目录)，local.conf(根据MACHINE,DISTRO自动生成)
    │    ├──── sstate-cache
    │    └──── tmp #编译过程文件
    ├──── dl    #预先下载的软件压缩包，如果没有则在编译时联网下载
    └──── sources
      ├──── base #配置文件
      ├──── meta-openembedded #原生yocto提供的扩展层，包含较多常用的layer及recipes
      ├──── meta-tina  #Tina yocto相关的meta包
      └──── poky  #原生yocto提供的核心层，包含基本的构建系统，必要的layer, recipes等，当前使用的是5.1版本styhead
```

9.5.2 编译环境

支持的编译主机环境说明参考如下，建议使用 ubuntu20 及以上版本。

```
主机支持列表：
https://docs.yoctoproject.org/ref-manual/system-requirements.html#supported-linux-distributions

关键工具版本：
Git 1.8.3.1 or greater
tar 1.28 or greater
Python 3.8.0 or greater.
gcc 8.0 or greater.
GNU make 4.0 or greater

相关软件包安装：
$ sudo apt install gawk wget git diffstat unzip texinfo gcc build-essential chrpath socat cpio python3 python3-pip
        python3-pexpect xz-utils debianutils iputils-ping python3-git python3-jinja2 python3-subunit zstd liblz4-tool file
        locales libacl1
$ sudo locale-gen en_US.UTF-8
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

yocto 官方的部署编译等说明可参考：https://docs.yoctoproject.org/brief-yoctoprojectqs/index.html

9.5.3 编译说明

9.5.3.1 方式一: 原始编译方式 (T527 为例)

```
#1. 配置(配置machine以及distro，生成build目录(关键生成build/conf/local.conf文件)

进入yocto根目录：

MACHINE=t527-demo DISTRO=tina-industry  source setup-environment build

#2. 编译生成rootfs

进入yocto根目录：

bitbake tina-image-core 或者 bitbake tina-image-minimal

生成的rootfs镜像路径：yocto/build/tmp/deploy/images/t527-demo/

ext4文件系统格式：tina-image-core-t527-demo.rootfs.ext4

squashfs文件系统格式：tina-image-core-t527-demo.rootfs.squashfs-xz

#3. 关联rootfs

进入AIOT-LINUX根目录

./build.sh yocto_rootfs

#4. 打包

进入AIOT-LINUX根目录

./build.sh pack
```

说明

原生 Yocto 快速部署编译说明可参考：https://docs.yoctoproject.org/brief-yoctoprojectqs/index.html

9.5.3.2 方式二: AW 全量编译方式 (T527 为例)

```
1. 配置
$ ./build.sh config
========ACTION List: mk_config ;========
options :
All available platform: //选择linux系统
  0. android
  1. linux
Choice [linux]:1
All available linux_dev: //选择yocto
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
  0. bsp
  1. dragonboard
  2. buildroot
  3. debian
  4. yocto
Choice [buildroot]:4
All available ic: //选择t527
  0. a523
  1. a527
  2. a733
  3. t527
  4. T736
Choice [t527]: 3
All available board: //选择demo_linux_aiot
  0. demo_linux_aiot
  1. demo_linux_aiot_nand
  2. demo_linux_aiot_spinand
Choice [demo_linux_aiot]:0
All available flash:  //选择default
  0. default
  1. nor
Choice [default]:0
All available kern_name: //选择内核版本 linux-5.15
  0. linux-5.10
  1. linux-5.15
Choice [linux-5.15]:1
Setup BSP files

2. 编译
./build.sh

3. 打包
./build.sh pack
```

说明

推荐使用编译方式:全量自动编译编译，

9.5.4 常用配置

9.5.4.1 machine 配置

配置放置路径：yocto/sources/meta-tina/conf/machine/

可配置波特率、串口等信息；

9.5.4.2 distro 配置

配置放置路径：yocto/sources/meta-tina/conf/yocto/ 发行版策略配置，工具链、init 等配置例如可设置 c 库选择 musl 或者 glibc

目前默认有 tina-industry,tina-minimal,tina-robot 几种配置 (暂时未有差异化)

版权所有 © 珠海全志科技股份有限公司。保留一切权利

9.5.4.3 image 配置

配置放置路径：yocto/sources/meta-tina/recipes-core/images

image 配置决定 rootfs 镜像中有哪些软件包目前默认有下面几种 image:

image 名字	描述
tina-image-core	添加了常用的 adbd,alsa-lib 等软件包
tina-image-minimal	最小系统，几乎没有任何软件包

9.5.5 常用快捷命令

命令	描述
cyroot	跳转 yocto 根目录
cybuild	跳转 yocto build 目录
cyout	跳转 yocto 镜像生成目录，包含有最终 rootfs 镜像
cyrootfs	跳转 yocto rootfs 目录，最终 rootfs 镜像的来源
cytina	跳转 meta-tina 目录
cypoky	跳转 poky 目录
cymachine	跳转存放 machine 配置的目录
cydistro	跳转存放 distro 配置的目录

说明

需要执行下面命令加载环境变量后，才能使用快速跳转命令：source build/envsetup.sh

9.6 openwrt 开发

待补充

9.7 Recovery 开发

recovery 系统方案，是在主系统之外，增加一个 recovery 系统。升级时，主系统负责升级 recovery 系统，recovery 系统负责升级主系统。这样如果升级中途发生掉电，也不会影响当前正在使用的这个系统。重启后仍可正常进入系统，继续完成升级。一般 recovery 系统会使用 initramfs 功能，并大量裁剪不必要的应用，只保留 OTA 必需的功能，把 size 尽量减小。

recovery 系统方案优点：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- recovery 系统可以做得比较小，省 flash 空间。

recovery 系统方案缺点：

- recovery 系统一般不包含主应用，所以 OTA 期间，处于 recovery 系统中时，无法为用户正常提供服务。
- 需要重启两次。
- 需要维护两份系统配置，即主系统和 recovery 系统。

关于 Recovery 开发更详细的说明，请参考如下文档：

Software软件类文档/基础组件开发指南/Recovery/《Tina_Linux_OTA_开发指南.pdf》

9.8 多媒体开发

目前全志在 gstreamer 通路上对接了硬件编解码，因此 app 可以基于 gstreamer 的 pipeline 方式，来进行完整应用的开发。

gstreamer 向下对接全志平台视频编解码中间件 libcedarc。libcedarc 是编解码硬件引擎的硬件抽象层，开发者也可自行基于 libcedarc 的接口对接硬件编解码模块实现更加灵活的功能。整体框图如下所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

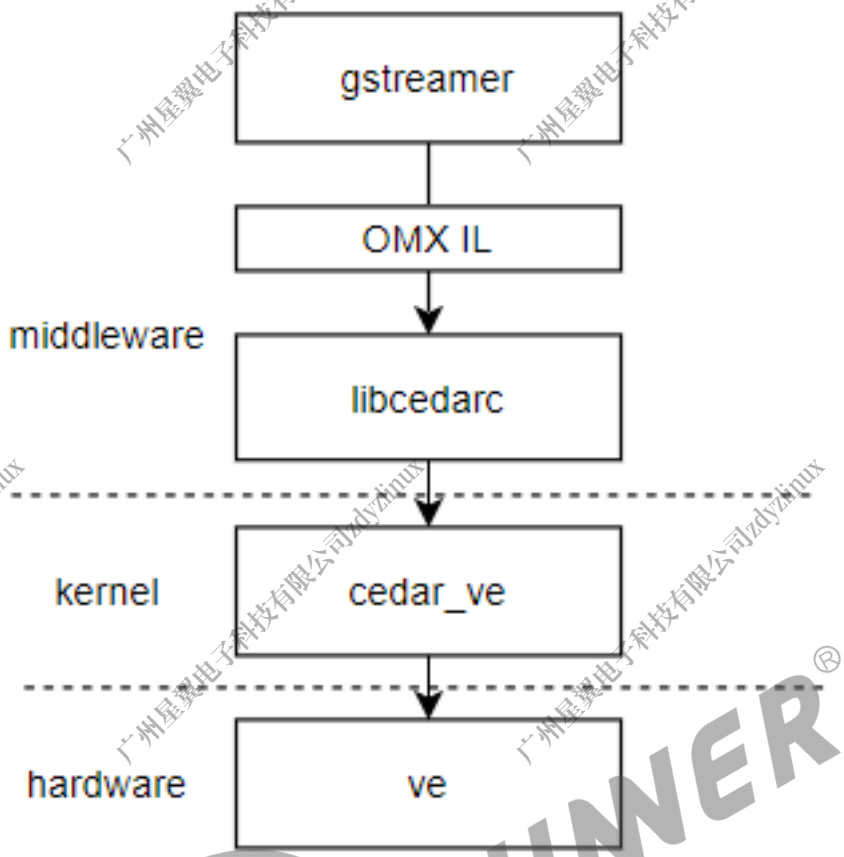

图 9-1: 多媒体框架图

debian 系统上，rootfs 中默认预置了 gstreamer 插件库以及 libcedarc 的库，如需基于 libcedarc 进行开发验证，可以将sdkroot/debian/packages/arm64目录下的 **libcedarc-dev_1.0.0_arm64.deb** 安装到 debian 系统下，其内部包含编解码测试用例和开发所需头文件。

关于 libcedarc 以及 gstreamer 更详细的说明，请参考如下文档：

```
Software 软件类文档/基础组件开发指南/Multimedia/《Linux_CedarE_开发指南.pdf》
Software 软件类文档/基础组件开发指南/Multimedia/《Linux_CedarX_开发指南.pdf》
Software 软件类文档/基础组件开发指南/Multimedia/《Linux_Decoder_开发指南.pdf》
Software 软件类文档/基础组件开发指南/Multimedia/《Linux_Encoder_开发指南.pdf》
Software 软件类文档/基础组件开发指南/Multimedia/《Linux_Gstreamer_开发指南.pdf》
```

9.9 图显开发

allwinner Aiot Linux 平台的图显，采用的是一个较为通用的架构，是应用 DRM 和 DMA-BUF 的 ARM Linux 平台。

更多的资料参考：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

Software 软件类文档\SDK模块开发指南\DISPLAY (DRM)\《AW_DRM屏幕适配_调试指南.pdf》
Software 软件类文档\SDK模块开发指南\DISPLAY (DRM)\《AW_User_Guide_Linux_DRM_CN.pdf》
Software 软件类文档\SDK模块开发指南\DISPLAY (DRM)\《Linux_HDMI20_DRM_开发指南.pdf》

Software 软件类文档\基础组件开发指南\Graphics\《AW_Developer_Guide_Buildroot_Weston_CN.pdf》
Software 软件类文档\基础组件开发指南\Graphics\《AW_Developer_Guide_Linux_Graphics_CN.pdf》

9.10 安全机制开发

详情参考目录文档：

Software 软件类文档\基础组件开发指南\SECURE

9.11 应用开发

SDK 常用的应用有 Weston、ROS2 等应用开发，参考目录文档：

目录：Software 软件类文档\基础组件开发指南\<ApplicationNote>
目录：Software 软件类文档\基础组件开发指南\<ROS2>

9.12 WIFI/BT 开发

详情见：Software 软件类文档\基础组件开发指南\Wifibt

9.13 TP/Sensor 开发

待补充

版权所有 © 珠海全志科技股份有限公司。保留一切权利

10 SDK 测试

待补充

广州星翼电子科技有限公司zdyzlinux

11 SDK 调试

11.1 ADB 工具

11.1.1 概述

- 运行设备的 shell（命令行）
- 管理模拟器或设备的端口映射
- 计算机和设备之间上传/下载文件
- 将本地软件安装至板卡设备中
- ADB 是一个 “客户端—服务器端” 程序，其中客户端主要是指 PC，服务器端是目标硬件板卡(如 Debian 设备的实体机器或者虚拟机)。根据 PC 连接目标硬件板卡设备的方式不同，ADB 可以分为两类：

网络 ADB：主机通过有线/无线网络（同一局域网）连接到 STB 设备 **USB ADB**：主机通过 USB 线连接到 STB 设备

11.1.2 USB adb 使用说明

USB adb 使用有以下限制：

- 只支持 USB OTG 口
- 不支持多个客户端同时使用
- 只支持主机连接一个设备，不支持连接多个设备

连接步骤如下：测试是否连接成功，运行”adb devices” 命令，如果显示机器的序列号，表示连接成功。

11.2 版权检测工具

11.2.1 Buildroot

待补充

版权所有 © 珠海全志科技股份有限公司。保留一切权利

11.2.2 Debian

待补充

11.2.3 yocto

待补充

版权所有 © 珠海全志科技股份有限公司。保留一切权利

12 常见问题 (FAQ)

12.1 如何查看系统相关信息

12.1.1 如何查看系统 Debian 版本

```
root@localhost:~# cat /etc/debian_version
11.11
```

12.1.2 如何查看 Debian 显示使用 X11 还是 wayland

执行命令：

```
root@localhost:~# echo $XDG_SESSION_TYPE
X11
```

表示使用：X11

```
root@localhost:~# echo $XDG_SESSION_TYPE
wayland
```

表示使用：wayland

12.1.3 如何查看系统分区情况

```
root@localhost:~# parted -l
Model: MMC AT3SGC (sd/mmc)
Disk /dev/mmcblk0: 62.5GB
Sector size (logical/physical): 512B/512B
Partition Table: gpt
Disk Flags:

Number  Start   End     Size    File system  Name        Flags
 1      37.7MB  71.3MB  33.6MB  fat16        boot-resource  msftdata
 2      71.3MB  88.1MB  16.8MB               env            msftdata
 3      88.1MB  189MB   101MB                boot           msftdata
 4      189MB   7991MB  7803MB  ext4         rootfs         msftdata
 5      7991MB  62.5GB  54.5GB               userdata       msftdata
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

12.2 Debian deb 包的制作，解压，修改，安装

```
#解压包里的内容到extract目录下
dpkg -X xxx.deb extract/

#解压出包里的控制信息到extrack/DEBIAN/目录下：
dpkg -e xxx.deb extract/DEBIAN/

#修改文件xxx


# 对修改后文件重新打包生成deb包：new-xxx.deb
dpkg-deb -b extract/ ./new-xxx.deb

#安装deb包
dpkg -i new-xxx.deb
```

安装过程，若出现如下报错，可以使用–force-overwrite 来强制覆盖

```
root@localhost:/# dpkg -i xserver-xorg-mesa-g57_1.21.1-2.deb
(Reading database ... 61175 files and directories currently installed.)
Preparing to unpack xserver-xorg-mesa-g57_1.21.1-2.deb ...
Unpacking sunxi-xorg-deb (1.0.0) ...
dpkg: error processing archive xserver-xorg-mesa-g57_1.21.1-2.deb (--install):
 trying to overwrite '/usr/lib/xorg/modules/extensions/libglx.so', which is also in package xserver-xorg-core 2:1.20.11-1+
        deb11u13
dpkg-deb: error: paste subprocess was killed by signal (Broken pipe)
Errors were encountered while processing:
 xserver-xorg-mesa-g57_1.21.1-2.deb
```

```
root@localhost:/# dpkg -i --force-overwrite xserver-xorg-mesa-g57_1.21.1-2.deb

(Reading database ... 61175 files and directories currently installed.)
Preparing to unpack xserver-xorg-mesa-g57_1.21.1-2.deb ...
Unpacking sunxi-xorg-deb (1.0.0) ...
dpkg: warning: overriding problem because --force enabled:
dpkg: warning: trying to overwrite '/usr/lib/xorg/modules/extensions/libglx.so', which is also in package xserver-xorg-
        core 2:1.20.11-1+deb11u13
dpkg: warning: overriding problem because --force enabled:
dpkg: warning: trying to overwrite '/usr/lib/xorg/modules/drivers/modesetting_drv.so', which is also in package xserver-
        xorg-core 2:1.20.11-1+deb11u13
dpkg: warning: overriding problem because --force enabled:
dpkg: warning: trying to overwrite '/usr/bin/Xorg', which is also in package xserver-xorg-core 2:1.20.11-1+deb11u13
Setting up sunxi-xorg-deb (1.0.0) ...
```

12.3 Buildroot 如何对特定软件包进行特定操作

```
#buildroot特定软件包进行特定操作

./build.sh buildroot_package <pkg>-<operate>


# 在根目录下，<pkg>即为软件包名字
./build.sh buildroot_package <pkg>          - Build and install <pkg> and all its dependencies----------单独编译对
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
应APP
./build.sh buildroot_package <pkg>-source        - Only download the source files for <pkg>
./build.sh buildroot_package <pkg>-extract       - Extract <pkg> sources
./build.sh buildroot_package <pkg>-patch         - Apply patches to <pkg>
./build.sh buildroot_package <pkg>-depends       - Build <pkg>'s dependencies
./build.sh buildroot_package <pkg>-configure     - Build <pkg> up to the configure step
./build.sh buildroot_package <pkg>-build         - Build <pkg> up to the build step
./build.sh buildroot_package <pkg>-show-depends  - List packages on which <pkg> depends
./build.sh buildroot_package <pkg>-show-rdepends - List packages which have <pkg> as a dependency
./build.sh buildroot_package <pkg>-graph-depends - Generate a graph of <pkg>'s dependencies
./build.sh buildroot_package <pkg>-graph-rdepends - Generate a graph of <pkg>'s reverse dependencies
./build.sh buildroot_package <pkg>-dirclean      - Remove <pkg> build directory----------清除对应
    APP的编译目录
./build.sh buildroot_package <pkg>-reconfigure   - Restart the build from the configure step
./build.sh buildroot_package <pkg>-rebuild       - Restart the build from the build step----------单独重新
    编译对应APP
```

12.4 如何新增 yocto 方案 (以 T527 为例)

```
#1.device芯片配置仓库下，板级方案中添加yocto目录及配置

device/config/chips/t527/configs/demo_linux_aiot下新建yocto目录：

yocto/
├── BoardConfig.mk
├── env.cfg
└── sys_partition.fex

a. BoardConfig.mk中需要指定LICHEE_YOCTO_MACHINE，LICHEE_YOCTO_DISTRO，LICHEE_YOCTO_IMAGE三个变量，决
    定使用yocto中哪些machine,distro,image
b.env.cfg中注意init的路径(当前yocto配置默认使用busybox init)：init=/sbin/init
c.sys_partition.fex根据实际空间进行配置即可，无特殊要求

t527下的BoardConfig.mk：
LICHEE_YOCTO_MACHINE:=t527-demo
LICHEE_YOCTO_DISTRO:=tina-industry
LICHEE_YOCTO_IMAGE:=tina-image-core

#2.新增machine配置

yocto/sources/meta-tina/conf/machine/t527-demo.conf

参考复制一份即可

一般情况更换machine配置即可。完成后./build.sh配置&编译即可
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。